10 minutes to pandas
Intro to data structures
Essential basic functionality
IO tools (text, CSV, HDF5, …)
PyArrow Functionality
Indexing and selecting data
MultiIndex / advanced indexing
Copy-on-Write (CoW)
Merge, join, concatenate and compare
Reshaping and pivot tables
Working with text data
Working with missing data
Duplicate Labels
Categorical data
Nullable integer data type
Nullable Boolean data type
Chart visualization
Table Visualization
User-Defined Functions (UDFs)
Group by: split-apply-combine
Windowing operations
Time series / date functionality
Time deltas

User Guide > Group by: split-apply-combine

On this page
Splitting an object into groups
Iterating through groups
Selecting a group
Aggregation
Transformation
Filtration
Flexible `apply`
Numba accelerated routines
Other useful features
Examples

Group by: split-apply-combine

By "group by" we are referring to a process involving one or more of the following steps:

- **Splitting** the data into groups based on some criteria.
- **Applying** a function to each group independently.
- **Combining** the results into a data structure.

Out of these, the split step is the most straightforward. In the apply step, we might wish to do one of the following:

- **Aggregation**: compute a summary statistic (or statistics) for each group. Some examples:
  - Compute group sums or means.
  - Compute group sizes / counts.
- **Transformation**: perform some group-specific computations and return a like-indexed object. Some examples:
  - Standardize data (zscore) within a group.
  - Filling NAs within groups with a value derived from each group.
- **Filtration**: discard some groups, according to a group-wise computation that evaluates to True or False. Some examples:
  - Discard data that belong to groups with only a few members.
  - Filter out data based on the group sum or mean.

Many of these operations are defined on GroupBy objects. These operations are similar to those of the aggregating API, window API, and resample API.

It is possible that a given operation does not fall into one of these categories or is some combination of them. In such a case, it may be possible to compute the operation using GroupBy's `apply` method. This method will examine the results of the apply step and try to sensibly combine them into a single result if it doesn't fit into either of the above three categories.

> **Note**
>
> An operation that is split into multiple steps using built-in GroupBy operations will be more efficient than using the `apply` method with a user-defined Python function.

The name GroupBy should be quite familiar to those who have used a SQL-based tool (or `itertools`), in which you can write code like:

```
SELECT Column1, Column2, mean(Column3), sum(Column4)
FROM SomeTable
GROUP BY Column1, Column2
```

We aim to make operations like this natural and easy to express using pandas. We'll address each area of GroupBy functionality, then provide some non-trivial examples / use cases.

See the cookbook for some advanced strategies.

Splitting an object into groups

The abstract definition of grouping is to provide a mapping of labels to group names. To create a GroupBy object (more on what the GroupBy object is later), you may do the following:

```
In [1]: speeds = pd.DataFrame(
   ...:     [
   ...:         ("bird", "Falconiformes", 389.0),
   ...:         ("bird", "Psittaciformes", 24.0),
   ...:         ("mammal", "Carnivora", 80.2),
   ...:         ("mammal", "Primates", np.nan),
   ...:         ("mammal", "Carnivora", 58),
   ...:     ],
   ...:     index=["falcon", "parrot", "lion", "monkey", "leopard"],
   ...:     columns=("class", "order", "max_speed"),
   ...: )
   ...: 

In [2]: speeds
Out[2]: 
          class           order  max_speed
falcon     bird   Falconiformes      389.0
parrot     bird  Psittaciformes       24.0
lion     mammal       Carnivora       80.2
monkey   mammal        Primates        NaN
leopard  mammal       Carnivora       58.0

In [3]: grouped = speeds.groupby("class")

In [4]: grouped = speeds.groupby(["class", "order"])
```

The mapping can be specified many different ways:

- A Python function, to be called on each of the index labels.
- A list or NumPy array of the same length as the index.
- A dict or `Series`, providing a `label -> group name` mapping.
- For `DataFrame` objects, a string indicating either a column name or an index level name to be used to group.
- A list of any of the above things.

Collectively we refer to the grouping objects as the **keys**. For example, consider the following `DataFrame`:

> **Note**
>
> A string passed to `groupby` may refer to either a column or an index level. If a string matches both a column name and an index level name, a `ValueError` will be raised.

```
In [5]: df = pd.DataFrame(
   ...:     {
   ...:         "A": ["foo", "bar", "foo", "bar", "foo", "bar", "foo", "foo"],
   ...:         "B": ["one", "one", "two", "three", "two", "two", "one", "three"],
   ...:         "C": np.random.randn(8),
   ...:         "D": np.random.randn(8),
   ...:     }
   ...: )
   ...: 

In [6]: df
Out[6]: 
     A      B         C         D
0  foo    one  0.469112 -0.861849
1  bar    one -0.282863 -2.104569
2  foo    two -1.509059 -0.494929
3  bar  three -1.135632  1.071804
4  foo    two  1.212112  0.721555
5  bar    two -0.173215 -0.706771
6  foo    one  0.119209 -1.039575
7  foo  three -1.044236  0.271860
```

On a DataFrame, we obtain a GroupBy object by calling `groupby()`. This method returns a `pandas.api.typing.DataFrameGroupBy` instance. We could naturally group by either the `A` or `B` columns, or both:

```
In [7]: grouped = df.groupby("A")

In [8]: grouped = df.groupby("B")

In [9]: grouped = df.groupby(["A", "B"])
```

> **Note**
>
> `df.groupby('A')` is just syntactic sugar for `df.groupby(df['A'])`.

DataFrame groupby always operates along axis 0 (rows). To split by columns, first do a transpose:

```
In [10]: def get_letter_type(letter):
   ....:     if letter.lower() in 'aeiou':
   ....:         return 'vowel'
   ....:     else:
   ....:         return 'consonant'
   ....: 

In [11]: grouped = df.T.groupby(get_letter_type)
```

pandas `Index` objects support duplicate values. If a non-unique index is used as the group key in a groupby operation, all values for the same index value will be considered to be in one group and thus the output of aggregation functions will only contain unique index values:

```
In [12]: index = [1, 2, 3, 1, 2, 3]

In [13]: s = pd.Series([1, 2, 3, 10, 20, 30], index=index)

In [14]: s
Out[14]: 
1     1
2     2
3     3
1    10
2    20
3    30
dtype: int64

In [15]: grouped = s.groupby(level=0)

In [16]: grouped.first()
Out[16]: 
1    1
2    2
3    3
dtype: int64

In [17]: grouped.last()
Out[17]: 
1    10
2    20
3    30
dtype: int64

In [18]: grouped.sum()
Out[18]: 
1    11
2    22
3    33
dtype: int64
```

Note that **no splitting occurs** until it's needed. Creating the GroupBy object only verifies that you've passed a valid mapping.

> **Note**
>
> Many kinds of complicated data manipulations can be expressed in terms of GroupBy operations (though it can't be guaranteed to be the most efficient implementation). You can get quite creative with the label mapping functions.

GroupBy sorting

By default the group keys are sorted during the `groupby` operation. You may however pass `sort=False` for potential speedups. With `sort=False` the order among group-keys follows the order of appearance of the keys in the original dataframe:

```
In [19]: df2 = pd.DataFrame({"X": ["B", "B", "A", "A"], "Y": [1, 2, 3, 4]})

In [20]: df2.groupby(["X"]).sum()
Out[20]: 
   Y
X   
A  7
B  3

In [21]: df2.groupby(["X"], sort=False).sum()
Out[21]: 
   Y
X   
B  3
A  7
```

Note that `groupby` will preserve the order in which *observations* are sorted *within* each group. For example, the groups created by `groupby()` below are in the order they appeared in the original `DataFrame`:

```
In [22]: df3 = pd.DataFrame({"X": ["A", "B", "A", "B"], "Y": [1, 4, 3, 2]})

In [23]: df3.groupby("X").get_group("A")
Out[23]: 
   X  Y
0  A  1
2  A  3

In [24]: df3.groupby(["X"]).get_group(("B",))
Out[24]: 
   X  Y
1  B  4
3  B  2
```

GroupBy dropna

By default `NA` values are excluded from group keys during the `groupby` operation. However, in case you want to include `NA` values in group keys, you could pass `dropna=False` to achieve it.

```
In [25]: df_list = [[1, 2, 3], [1, None, 4], [2, 1, 3], [1, 2, 2]]

In [26]: df_dropna = pd.DataFrame(df_list, columns=["a", "b", "c"])

In [27]: df_dropna
Out[27]: 
   a    b  c
0  1  2.0  3
1  1  NaN  4
2  2  1.0  3
3  1  2.0  2
```

```
# Default ``dropna`` is set to True, which will exclude NaNs in keys
In [28]: df_dropna.groupby(by=["b"], dropna=True).sum()
Out[28]: 
     a  c
b        
1.0  2  3
2.0  2  5

# In order to allow NaN in keys, set ``dropna`` to False
In [29]: df_dropna.groupby(by=["b"], dropna=False).sum()
Out[29]: 
     a  c
b        
1.0  2  3
2.0  2  5
NaN  1  4
```

The default setting of `dropna` argument is `True` which means `NA` are not included in group keys.

GroupBy object attributes

The `groups` attribute of a GroupBy object is a dictionary that maps each unique group key to the index labels belonging to that group. In the above example:

```
In [30]: df.groupby("A").groups
Out[30]: {'bar': [1, 3, 5], 'foo': [0, 2, 4, 6, 7]}

In [31]: df.T.groupby(get_letter_type).groups
Out[31]: {'consonant': ['B', 'C', 'D'], 'vowel': ['A']}
```

Calling the standard Python `len` function on the GroupBy object returns the number of groups, which is the same as the length of the `groups` dictionary:

```
In [32]: grouped = df.groupby(["A", "B"])

In [33]: grouped.groups
Out[33]: 
{('bar', 'one'): RangeIndex(start=1, stop=2, step=1),
 ('bar', 'three'): RangeIndex(start=3, stop=4, step=1),
 ('bar', 'two'): RangeIndex(start=5, stop=6, step=1),
 ('foo', 'one'): RangeIndex(start=0, stop=12, step=6),
 ('foo', 'three'): RangeIndex(start=7, stop=8, step=1),
 ('foo', 'two'): RangeIndex(start=2, stop=6, step=2)}

In [34]: len(grouped)
Out[34]: 6
```

`GroupBy` will tab complete column names, GroupBy operations, and other attributes:

```
In [35]: n = 10

In [36]: weight = np.random.normal(166, 20, size=n)

In [37]: height = np.random.normal(60, 10, size=n)

In [38]: time = pd.date_range("1/1/2000", periods=n)

In [39]: gender = np.random.choice(["male", "female"], size=n)

In [40]: df = pd.DataFrame(
   ....:     {"height": height, "weight": weight, "gender": gender}, index=time
   ....: )
   ....: 

In [41]: df
Out[41]: 
               height      weight  gender
2000-01-01  42.849980  157.500553    male
2000-01-02  49.607315  177.340407    male
2000-01-03  56.293531  171.524640    male
2000-01-04  48.421077  144.251986  female
2000-01-05  46.556882  152.526206    male
2000-01-06  68.448851  168.272968  female
2000-01-07  70.757698  136.431469    male
2000-01-08  58.909500  176.499753  female
2000-01-09  76.435631  174.094104  female
2000-01-10  45.306120  177.540920    male

In [42]: gb = df.groupby("gender")
```

```
In [43]: gb.<TAB>  # noqa: E225, E999
gb.agg        gb.boxplot    gb.cummin     gb.describe   gb.filter     gb.get_group
gb.aggregate  gb.count      gb.cumprod    gb.dtype      gb.first      gb.groups
gb.apply      gb.cummax     gb.cumsum     gb.gender     gb.head       gb.indices
```

GroupBy with MultiIndex

With hierarchically-indexed data, it's quite natural to group by one of the levels of the hierarchy.

Let's create a Series with a two-level `MultiIndex`.

```
In [44]: arrays = [
   ....:     ["bar", "bar", "baz", "baz", "foo", "foo", "qux", "qux"],
   ....:     ["one", "two", "one", "two", "one", "two", "one", "two"],
   ....: ]
   ....: 

In [45]: index = pd.MultiIndex.from_arrays(arrays, names=["first", "second"])

In [46]: s = pd.Series(np.random.randn(8), index=index)

In [47]: s
Out[47]: 
first  second
bar    one      -0.919854
       two      -0.042379
baz    one       1.247642
       two      -0.009920
foo    one       0.290213
       two       0.495767
qux    one       0.362949
       two       1.548106
dtype: float64
```

We can then group by one of the levels in `s`.

```
In [48]: grouped = s.groupby(level=0)

In [49]: grouped.sum()
Out[49]: 
first
bar   -0.962232
baz    1.237723
foo    0.785980
qux    1.911055
dtype: float64
```

If the MultiIndex has names specified, these can be passed instead of the level number:

```
In [50]: s.groupby(level="second").sum()
Out[50]: 
second
one    0.980950
two    1.991575
dtype: float64
```

Grouping with multiple levels is supported.

```
In [51]: arrays = [
   ....:     ["bar", "bar", "baz", "baz", "foo", "foo", "qux", "qux"],
   ....:     ["doo", "doo", "bee", "bee", "bop", "bop", "bop", "bop"],
   ....:     ["one", "two", "one", "two", "one", "two", "one", "two"],
   ....: ]
   ....: 

In [52]: index = pd.MultiIndex.from_arrays(arrays, names=["first", "second", "third"

In [53]: s = pd.Series(np.random.randn(8), index=index)

In [54]: s
Out[54]: 
first  second  third
bar    doo     one     -1.131345
               two     -0.089329
baz    bee     one      0.337863
               two     -0.945867
foo    bop     one     -0.932132
               two      1.956030
qux    bop     one      0.017587
               two     -0.016692
dtype: float64

In [55]: s.groupby(level=["first", "second"]).sum()
Out[55]: 
first  second
bar    doo      -1.220674
baz    bee      -0.608004
foo    bop       1.023898
qux    bop       0.000895
dtype: float64
```

Index level names may be supplied as keys.

```
In [56]: s.groupby(["first", "second"]).sum()
Out[56]: 
first  second
bar    doo      -1.220674
baz    bee      -0.608004
foo    bop       1.023898
qux    bop       0.000895
dtype: float64
```

More on the `sum` function and aggregation later.

Grouping DataFrame with Index levels and columns

A DataFrame may be grouped by a combination of columns and index levels. You can specify both column and index names, or use a `Grouper`.

Let's first create a DataFrame with a MultiIndex:

```
In [57]: arrays = [
   ....:     ["bar", "bar", "baz", "baz", "foo", "foo", "qux", "qux"],
   ....:     ["one", "two", "one", "two", "one", "two", "one", "two"],
   ....: ]
   ....: 

In [58]: index = pd.MultiIndex.from_arrays(arrays, names=["first", "second"])

In [59]: df = pd.DataFrame({"A": [1, 1, 1, 1, 2, 2, 3, 3], "B": np.arange(8)}, inde

In [60]: df
Out[60]: 
              A  B
first second      
bar   one     1  0
      two     1  1
baz   one     1  2
      two     1  3
foo   one     2  4
      two     2  5
qux   one     3  6
      two     3  7
```

Then we group `df` by the `second` index level and the `A` column.

```
In [61]: df.groupby([pd.Grouper(level=1), "A"]).sum()
Out[61]: 
          B
second A   
one    1  2
       2  4
       3  6
two    1  4
       2  5
       3  7
```

Index levels may also be specified by name.

```
In [62]: df.groupby([pd.Grouper(level="second"), "A"]).sum()
Out[62]: 
          B
second A   
one    1  2
       2  4
       3  6
two    1  4
       2  5
       3  7
```

Index level names may be specified as keys directly to `groupby`.

```
In [63]: df.groupby(["second", "A"]).sum()
Out[63]: 
          B
second A   
one    1  2
       2  4
       3  6
two    1  4
       2  5
       3  7
```

DataFrame column selection in GroupBy

Once you have created the GroupBy object from a DataFrame, you might want to do something different for each of the columns. Thus, by using `[]` on the GroupBy object in a similar way as the one used to get a column from a DataFrame, you can do:

```
In [64]: df = pd.DataFrame(
   ....:     {
   ....:         "A": ["foo", "bar", "foo", "bar", "foo", "bar", "foo", "foo"],
   ....:         "B": ["one", "one", "two", "three", "two", "two", "one", "three"],
   ....:         "C": np.random.randn(8),
   ....:         "D": np.random.randn(8),
   ....:     }
   ....: )
   ....: 

In [65]: df
Out[65]: 
     A      B         C         D
0  foo    one -0.575247  1.346061
1  bar    one  0.254161  1.511763
2  foo    two -1.143704  1.627081
3  bar  three  0.215897 -0.990582
4  foo    two  1.193555 -0.441652
5  bar    two -0.077118  1.211526
6  foo    one -0.408530  0.268520
7  foo  three -0.862495  0.024580

In [66]: grouped = df.groupby(["A"])

In [67]: grouped_C = grouped["C"]

In [68]: grouped_D = grouped["D"]
```

This is mainly syntactic sugar for the alternative, which is much more verbose:

```
In [69]: df["C"].groupby(df["A"])
Out[69]: <pandas.api.typing.SeriesGroupBy object at 0x7f5f18eaad50>
```

Additionally, this method avoids recomputing the internal grouping information derived from the passed key.

You can also include the grouping columns if you want to operate on them.

```
In [70]: grouped[["A", "B"]].sum()
Out[70]: 
                   A                  B
A                                      
bar        barbarbar        onethreetwo
foo  foofoofoofoofoo  onetwotwoonethree
```

> **Note**
>
> The `groupby` operation in pandas drops the `name` field of the columns Index object after the operation. This change ensures consistency in syntax between different column selection methods within groupby operations.

Iterating through groups

With the GroupBy object in hand, iterating through the grouped data is very natural and functions similarly to `itertools.groupby()`:

```
In [71]: grouped = df.groupby('A')

In [72]: for name, group in grouped:
   ....:     print(name)
   ....:     print(group)
   ....: 
bar
     A      B         C         D
1  bar    one  0.254161  1.511763
3  bar  three  0.215897 -0.990582
5  bar    two -0.077118  1.211526
foo
     A      B         C         D
0  foo    one -0.575247  1.346061
2  foo    two -1.143704  1.627081
4  foo    two  1.193555 -0.441652
6  foo    one -0.408530  0.268520
7  foo  three -0.862495  0.024580
```

In the case of grouping by multiple keys, the group name will be a tuple:

```
In [73]: for name, group in df.groupby(['A', 'B']):
   ....:     print(name)
   ....:     print(group)
   ....: 
('bar', 'one')
     A    B         C         D
1  bar  one  0.254161  1.511763
('bar', 'three')
     A      B         C         D
3  bar  three  0.215897 -0.990582
('bar', 'two')
     A    B         C         D
5  bar  two -0.077118  1.211526
('foo', 'one')
     A    B         C         D
0  foo  one -0.575247  1.346061
6  foo  one -0.408530  0.268520
('foo', 'three')
     A      B         C        D
7  foo  three -0.862495  0.02458
('foo', 'two')
     A    B         C         D
2  foo  two -1.143704  1.627081
4  foo  two  1.193555 -0.441652
```

See Iterating through groups.

Selecting a group

A single group can be selected using `DataFrameGroupBy.get_group()`:

```
In [74]: grouped.get_group("bar")
Out[74]: 
     A      B         C         D
1  bar    one  0.254161  1.511763
3  bar  three  0.215897 -0.990582
5  bar    two -0.077118  1.211526
```

Or for an object grouped on multiple columns:

```
In [75]: df.groupby(["A", "B"]).get_group(("bar", "one"))
Out[75]: 
     A    B         C         D
1  bar  one  0.254161  1.511763
```

Aggregation

An aggregation is a GroupBy operation that reduces the dimension of the grouping object. The result of an aggregation is, or at least is treated as, a scalar value for each column in a group. For example, producing the sum of each column in a group of values.

```
In [76]: animals = pd.DataFrame(
   ....:     {
   ....:         "kind": ["cat", "dog", "cat", "dog"],
   ....:         "height": [9.1, 6.0, 9.5, 34.0],
   ....:         "weight": [7.9, 7.5, 9.9, 198.0],
   ....:     }
   ....: )
   ....: 

In [77]: animals
Out[77]: 
  kind  height  weight
0  cat     9.1     7.9
1  dog     6.0     7.5
2  cat     9.5     9.9
3  dog    34.0   198.0

In [78]: animals.groupby("kind").sum()
Out[78]: 
      height  weight
kind                
cat     18.6    17.8
dog     40.0   205.5
```

In the result, the keys of the groups appear in the index by default. They can be instead included in the columns by passing `as_index=False`.

```
In [79]: animals.groupby("kind", as_index=False).sum()
Out[79]: 
  kind  height  weight
0  cat    18.6    17.8
1  dog    40.0   205.5
```

Built-in aggregation methods

Many common aggregations are built-in to GroupBy objects as methods. Of the methods listed below, those with a `*` do *not* have an efficient, GroupBy-specific, implementation.

Method	Description
`any()`	Compute whether any of the values in the groups are truthy
`all()`	Compute whether all of the values in the groups are truthy
`count()`	Compute the number of non-NA values in the groups
`cov()` *	Compute the covariance of the groups
`first()`	Compute the first occurring value in each group
`idxmax()`	Compute the index of the maximum value in each group
`idxmin()`	Compute the index of the minimum value in each group
`last()`	Compute the last occurring value in each group
`max()`	Compute the maximum value in each group
`mean()`	Compute the mean of each group
`median()`	Compute the median of each group
`min()`	Compute the minimum value in each group
`nunique()`	Compute the number of unique values in each group
`prod()`	Compute the product of the values in each group
`quantile()`	Compute a given quantile of the values in each group
`sem()`	Compute the standard error of the mean of the values in each group
`size()`	Compute the number of values in each group
`skew()` *	Compute the skew of the values in each group
`std()`	Compute the standard deviation of the values in each group
`sum()`	Compute the sum of the values in each group
`var()`	Compute the variance of the values in each group

Some examples:

```
In [80]: df.groupby("A")[["C", "D"]].max()
Out[80]: 
            C         D
A                      
bar  0.254161  1.511763
foo  1.193555  1.627081

In [81]: df.groupby(["A", "B"]).mean()
Out[81]: 
                  C         D
A   B                        
bar one    0.254161  1.511763
    three  0.215897 -0.990582
    two   -0.077118  1.211526
foo one   -0.491888  0.807291
    three -0.862495  0.024580
    two    0.024925  0.592714
```

Another aggregation example is to compute the size of each group. This is included in GroupBy as the `size` method. It returns a Series whose index consists of the group names and the values are the sizes of each group.

```
In [82]: grouped = df.groupby(["A", "B"])

In [83]: grouped.size()
Out[83]: 
A    B    
bar  one      1
     three    1
     two      1
foo  one      2
     three    1
     two      2
dtype: int64
```

While the `DataFrameGroupBy.describe()` method is not itself a reducer, it can be used to conveniently produce a collection of summary statistics about each of the groups.

```
In [84]: grouped.describe()
Out[84]: 
              C                      ...         D                    
          count      mean       std  ...       50%       75%       max
A   B                                ...                              
bar one     1.0  0.254161       NaN  ...  1.511763  1.511763  1.511763
    three   1.0  0.215897       NaN  ... -0.990582 -0.990582 -0.990582
    two     1.0 -0.077118       NaN  ...  1.211526  1.211526  1.211526
foo one     2.0 -0.491888  0.117887  ...  0.807291  1.076676  1.346061
    three   1.0 -0.862495       NaN  ...  0.024580  0.024580  0.024580
    two     2.0  0.024925  1.652692  ...  0.592714  1.109898  1.627081

[6 rows x 16 columns]
```

Another aggregation example is to compute the number of unique values of each group. This is similar to the `DataFrameGroupBy.value_counts()` function, except that it only counts the number of unique values.

```
In [85]: ll = [['foo', 1], ['foo', 2], ['foo', 2], ['bar', 1], ['bar', 1]]

In [86]: df4 = pd.DataFrame(ll, columns=["A", "B"])

In [87]: df4
Out[87]: 
     A  B
0  foo  1
1  foo  2
2  foo  2
3  bar  1
4  bar  1

In [88]: df4.groupby("A")["B"].nunique()
Out[88]: 
A
bar    1
foo    2
Name: B, dtype: int64
```

Note

Aggregation functions **will not** return the groups that you are aggregating over as named *columns* when `as_index=True`, the default. The grouped columns will be the **indices** of the returned object.

Passing `as_index=False` **will** return the groups that you are aggregating over as named columns, regardless if they are named **indices** or *columns* in the inputs.

The `aggregate()` method

Note

The `aggregate()` method can accept many different types of inputs. This section details using string aliases for various GroupBy methods; other inputs are detailed in the sections below.

Any reduction method that pandas implements can be passed as a string to `aggregate()`. Users are encouraged to use the shorthand, `agg`. It will operate as if the corresponding method was called.

```
In [89]: grouped = df.groupby("A")

In [90]: grouped[["C", "D"]].aggregate("sum")
Out[90]: 
            C         D
A                      
bar  0.392940  1.732707
foo -1.796421  2.824590

In [91]: grouped = df.groupby(["A", "B"])

In [92]: grouped.agg("sum")
Out[92]: 
                  C         D
A   B                        
bar one    0.254161  1.511763
    three  0.215897 -0.990582
    two   -0.077118  1.211526
foo one   -0.983776  1.614581
    three -0.862495  0.024580
    two    0.049851  1.185429
```

The result of the aggregation will have the group names as the new index. In the case of multiple keys, the result is a MultiIndex by default. As mentioned above, this can be changed by using the `as_index` option:

```
In [93]: grouped = df.groupby(["A", "B"], as_index=False)

In [94]: grouped.agg("sum")
Out[94]: 
     A      B         C         D
0  bar    one  0.254161  1.511763
1  bar  three  0.215897 -0.990582
2  bar    two -0.077118  1.211526
3  foo    one -0.983776  1.614581
4  foo  three -0.862495  0.024580
5  foo    two  0.049851  1.185429

In [95]: df.groupby("A", as_index=False)[["C", "D"]].agg("sum")
Out[95]: 
     A         C         D
0  bar  0.392940  1.732707
1  foo -1.796421  2.824590
```

Note that you could use the `DataFrame.reset_index()` DataFrame function to achieve the same result as the column names are stored in the resulting `MultiIndex`, although this will make an extra copy.

```
In [96]: df.groupby(["A", "B"]).agg("sum").reset_index()
Out[96]: 
     A      B         C         D
0  bar    one  0.254161  1.511763
1  bar  three  0.215897 -0.990582
2  bar    two -0.077118  1.211526
3  foo    one -0.983776  1.614581
4  foo  three -0.862495  0.024580
5  foo    two  0.049851  1.185429
```

Aggregation with user-defined functions

Users can also provide their own User-Defined Functions (UDFs) for custom aggregations.

Warning

When aggregating with a UDF, the UDF should not mutate the provided `Series`. See Mutating with User Defined Function (UDF) methods for more information.

Note

Aggregating with a UDF is often less performant than using the pandas built-in methods on GroupBy. Consider breaking up a complex operation into a chain of operations that utilize the built-in methods.

```
In [97]: animals
Out[97]: 
  kind  height  weight
0  cat     9.1     7.9
1  dog     6.0     7.5
2  cat     9.5     9.9
3  dog    34.0   198.0

In [98]: animals.groupby("kind")[["height"]].agg(lambda x: set(x))
Out[98]: 
           height
kind             
cat    {9.1, 9.5}
dog   {34.0, 6.0}
```

The resulting dtype will reflect that of the aggregating function. If the results from different groups have different dtypes, then a common dtype will be determined in the same way as `DataFrame` construction.

```
In [99]: animals.groupby("kind")[["height"]].agg(lambda x: x.astype(int).sum())
Out[99]: 
      height
kind        
cat       18
dog       40
```

Applying multiple functions at once

On a grouped `Series`, you can pass a list or dict of functions to `SeriesGroupBy.agg()`, outputting a DataFrame:

```
In [100]: grouped = df.groupby("A")

In [101]: grouped["C"].agg(["sum", "mean", "std"])
Out[101]: 
          sum      mean       std
A                                
bar  0.392940  0.130980  0.181231
foo -1.796421 -0.359284  0.912265
```

On a grouped `DataFrame`, you can pass a list of functions to `DataFrameGroupBy.agg()` to aggregate each column, which produces an aggregated result with a hierarchical column index:

```
In [102]: grouped[["C", "D"]].agg(["sum", "mean", "std"])
Out[102]: 
            C                             D                    
          sum      mean       std       sum      mean       std
A                                                              
bar  0.392940  0.130980  0.181231  1.732707  0.577569  1.366330
foo -1.796421 -0.359284  0.912265  2.824590  0.564918  0.884785
```

The resulting aggregations are named after the functions themselves.

For a `Series`, if you need to rename, you can add in a chained operation like this:

```
In [103]: (
   .....:     grouped["C"]
   .....:     .agg(["sum", "mean", "std"])
   .....:     .rename(columns={"sum": "foo", "mean": "bar", "std": "baz"})
   .....: )
   .....: 
Out[103]: 
          foo       bar       baz
A                                
bar  0.392940  0.130980  0.181231
foo -1.796421 -0.359284  0.912265
```

Or, you can simply pass a list of tuples each with the name of the new column and the aggregate function:

```
In [104]: (
   .....:    grouped["C"]
   .....:    .agg([("foo", "sum"), ("bar", "mean"), ("baz", "std")])
   .....: )
   .....: 
Out[104]: 
          foo       bar       baz
A                                
bar  0.392940  0.130980  0.181231
foo -1.796421 -0.359284  0.912265
```

For a grouped `DataFrame`, you can rename in a similar manner:

By chaining `rename` operation,

```
In [105]: (
   .....:     grouped[["C", "D"]].agg(["sum", "mean", "std"]).rename(
   .....:         columns={"sum": "foo", "mean": "bar", "std": "baz"}
   .....:     )
   .....: )
   .....: 
Out[105]: 
            C                             D                    
          foo       bar       baz       foo       bar       baz
A                                                              
bar  0.392940  0.130980  0.181231  1.732707  0.577569  1.366330
foo -1.796421 -0.359284  0.912265  2.824590  0.564918  0.884785
```

Or, passing a list of tuples,

```
In [106]: (
   .....:    grouped[["C", "D"]].agg(
   .....:        [("foo", "sum"), ("bar", "mean"), ("baz", "std")]
   .....:    )
   .....: )
   .....: 
Out[106]: 
            C                             D                    
          foo       bar       baz       foo       bar       baz
A                                                              
bar  0.392940  0.130980  0.181231  1.732707  0.577569  1.366330
foo -1.796421 -0.359284  0.912265  2.824590  0.564918  0.884785
```

Note

In general, the output column names should be unique, but pandas will allow you apply to the same function (or two functions with the same name) to the same column.

```
In [107]: grouped["C"].agg(["sum", "sum"])
Out[107]: 
          sum       sum
A                      
bar  0.392940  0.392940
foo -1.796421 -1.796421
```

pandas also allows you to provide multiple lambdas. In this case, pandas will mangle the name of the (nameless) lambda functions, appending `_<i>` to each subsequent lambda.

```
In [108]: grouped["C"].agg([lambda x: x.max() - x.min(), lambda x: x.median
Out[108]: 
     <lambda_0>  <lambda_1>
A                          
bar    0.331279    0.084917
foo    2.337259   -0.215962
```

Named aggregation

To support column-specific aggregation *with control over the output column names*, pandas accepts the special syntax in `DataFrameGroupBy.agg()` and `SeriesGroupBy.agg()`, known as "named aggregation", where

- The keywords are the *output* column names
- The values are tuples whose first element is the column to select and the second element is the aggregation to apply to that column. pandas provides the `NamedAgg` namedtuple with the fields `['column', 'aggfunc']` to make it clearer what the arguments are. As usual, the aggregation can be a callable or a string alias.

```
In [109]: animals
Out[109]: 
  kind  height  weight
0  cat     9.1     7.9
1  dog     6.0     7.5
2  cat     9.5     9.9
3  dog    34.0   198.0

In [110]: animals.groupby("kind").agg(
   .....:     min_height=pd.NamedAgg(column="height", aggfunc="min"),
   .....:     max_height=pd.NamedAgg(column="height", aggfunc="max"),
   .....:     average_weight=pd.NamedAgg(column="weight", aggfunc="mean"),
   .....: )
   .....: 
Out[110]: 
      min_height  max_height  average_weight
kind                                        
cat          9.1         9.5            8.90
dog          6.0        34.0          102.75
```

`NamedAgg` is just a `namedtuple`. Plain tuples are allowed as well.

```
In [111]: animals.groupby("kind").agg(
   .....:     min_height=("height", "min"),
   .....:     max_height=("height", "max"),
   .....:     average_weight=("weight", "mean"),
   .....: )
   .....: 
Out[111]: 
      min_height  max_height  average_weight
kind                                        
cat          9.1         9.5            8.90
dog          6.0        34.0          102.75
```

If the column names you want are not valid Python keywords, construct a dictionary and unpack the keyword arguments

```
In [112]: animals.groupby("kind").agg(
   .....:     **{
   .....:         "total weight": pd.NamedAgg(column="weight", aggfunc="sum")
   .....:     }
   .....: )
   .....: 
Out[112]: 
      total weight
kind              
cat           17.8
dog          205.5
```

When using named aggregation, additional keyword arguments are not passed through to the aggregation functions; only pairs of `(column, aggfunc)` should be passed as `**kwargs`. If your aggregation functions require additional arguments, apply them partially with `functools.partial()`.

Named aggregation is also valid for Series groupby aggregations. In this case there's no column selection, so the values are just the functions.

```
In [113]: animals.groupby("kind").height.agg(
   .....:     min_height="min",
   .....:     max_height="max",
   .....: )
   .....: 
Out[113]: 
      min_height  max_height
kind                        
cat          9.1         9.5
dog          6.0        34.0
```

Applying different functions to DataFrame columns

By passing a dict to `aggregate` you can apply a different aggregation to the columns of a DataFrame:

```
In [114]: grouped.agg({"C": "sum", "D": lambda x: np.std(x, ddof=1)})
Out[114]: 
            C         D
A                      
bar  0.392940  1.366330
foo -1.796421  0.884785
```

The function names can also be strings. In order for a string to be valid it must be implemented on GroupBy:

```
In [115]: grouped.agg({"C": "sum", "D": "std"})
Out[115]: 
            C         D
A                      
bar  0.392940  1.366330
foo -1.796421  0.884785
```

Transformation

A transformation is a GroupBy operation whose result is indexed the same as the one being grouped. Common examples include `cumsum()` and `diff()`.

```
In [116]: speeds
Out[116]: 
          class           order  max_speed
falcon     bird   Falconiformes      389.0
parrot     bird  Psittaciformes       24.0
lion     mammal       Carnivora       80.2
monkey   mammal        Primates        NaN
leopard  mammal       Carnivora       58.0

In [117]: grouped = speeds.groupby("class")["max_speed"]

In [118]: grouped.cumsum()
Out[118]: 
falcon     389.0
parrot     413.0
lion        80.2
monkey       NaN
leopard    138.2
Name: max_speed, dtype: float64

In [119]: grouped.diff()
Out[119]: 
falcon       NaN
parrot    -365.0
lion         NaN
monkey       NaN
leopard      NaN
Name: max_speed, dtype: float64
```

Unlike aggregations, the groupings that are used to split the original object are not included in the result.

Note

Since transformations do not include the groupings that are used to split the result, the arguments `as_index` and `sort` in `DataFrame.groupby()` and `Series.groupby()` have no effect.

A common use of a transformation is to add the result back into the original DataFrame.

```
In [120]: result = speeds.copy()

In [121]: result["cumsum"] = grouped.cumsum()

In [122]: result["diff"] = grouped.diff()

In [123]: result
Out[123]: 
          class           order  max_speed  cumsum   diff
falcon     bird   Falconiformes      389.0   389.0    NaN
parrot     bird  Psittaciformes       24.0   413.0 -365.0
lion     mammal       Carnivora       80.2    80.2    NaN
monkey   mammal        Primates        NaN     NaN    NaN
leopard  mammal       Carnivora       58.0   138.2    NaN
```

Built-in transformation methods

The following methods on GroupBy act as transformations.

Method	Description
`bfill()`	Back fill NA values within each group
`cumcount()`	Compute the cumulative count within each group
`cummax()`	Compute the cumulative max within each group
`cummin()`	Compute the cumulative min within each group
`cumprod()`	Compute the cumulative product within each group
`cumsum()`	Compute the cumulative sum within each group
`diff()`	Compute the difference between adjacent values within each group
`ffill()`	Forward fill NA values within each group
`pct_change()`	Compute the percent change between adjacent values within each group
`rank()`	Compute the rank of each value within each group
`shift()`	Shift values up or down within each group

In addition, passing any built-in aggregation method as a string to `transform()` (see the next section) will broadcast the result across the group, producing a transformed result. If the aggregation method has an efficient implementation, this will be performant as well.

The `transform()` method

Similar to the aggregation method, the `transform()` method can accept string aliases to the built-in transformation methods in the previous section. It can *also* accept string aliases to the built-in aggregation methods. When an aggregation method is provided, the result will be broadcast across the group.

```
In [124]: speeds
Out[124]: 
          class           order  max_speed
falcon     bird   Falconiformes      389.0
parrot     bird  Psittaciformes       24.0
lion     mammal       Carnivora       80.2
monkey   mammal        Primates        NaN
leopard  mammal       Carnivora       58.0

In [125]: grouped = speeds.groupby("class")[["max_speed"]]

In [126]: grouped.transform("cumsum")
Out[126]: 
         max_speed
falcon       389.0
parrot       413.0
lion          80.2
monkey         NaN
leopard      138.2

In [127]: grouped.transform("sum")
Out[127]: 
         max_speed
falcon       413.0
parrot       413.0
lion         138.2
monkey       138.2
leopard      138.2
```

In addition to string aliases, the `transform()` method can also accept User-Defined Functions (UDFs). The UDF must:

- Return a result that is either the same size as the group chunk or broadcastable to the size of the group chunk (e.g., a scalar, `grouped.transform(lambda x: x.iloc[-1])`).
- Operate column-by-column on the group chunk. The transform is applied to the first group chunk using chunk.apply.
- Not perform in-place operations on the group chunk. Group chunks should be treated as immutable, and changes to a group chunk may produce unexpected results. See Mutating with User Defined Function (UDF) methods for more information.
- (Optionally) operates on all columns of the entire group chunk at once. If this is supported, a fast path is used starting from the *second* chunk.

Note

Transforming by supplying `transform` with a UDF is often less performant than using the built-in methods on GroupBy. Consider breaking up a complex operation into a chain of operations that utilize the built-in methods.

All of the examples in this section can be made more performant by calling built-in methods instead of using UDFs. See below for examples.

Changed in version 2.0.0: When using `.transform` on a grouped DataFrame and the transformation function returns a DataFrame, pandas now aligns the result's index with the input's index. You can call `.to_numpy()` within the transformation function to avoid alignment.

Similar to The aggregate() method, the resulting dtype will reflect that of the transformation function. If the results from different groups have different dtypes, then a common dtype will be determined in the same way as `DataFrame` construction.

Suppose we wish to standardize the data within each group:

```
In [128]: index = pd.date_range("10/1/1999", periods=1100)

In [129]: ts = pd.Series(np.random.normal(0.5, 2, 1100), index)

In [130]: ts = ts.rolling(window=100, min_periods=100).mean().dropna()

In [131]: ts.head()
Out[131]: 
2000-01-08    0.779333
2000-01-09    0.778852
2000-01-10    0.786476
2000-01-11    0.782797
2000-01-12    0.798110
Freq: D, dtype: float64

In [132]: ts.tail()
Out[132]: 
2002-09-30    0.660294
2002-10-01    0.631095
2002-10-02    0.673601
2002-10-03    0.709213
2002-10-04    0.719369
Freq: D, dtype: float64

In [133]: transformed = ts.groupby(lambda x: x.year).transform(
   .....:     lambda x: (x - x.mean()) / x.std()
   .....: )
   .....: 
```

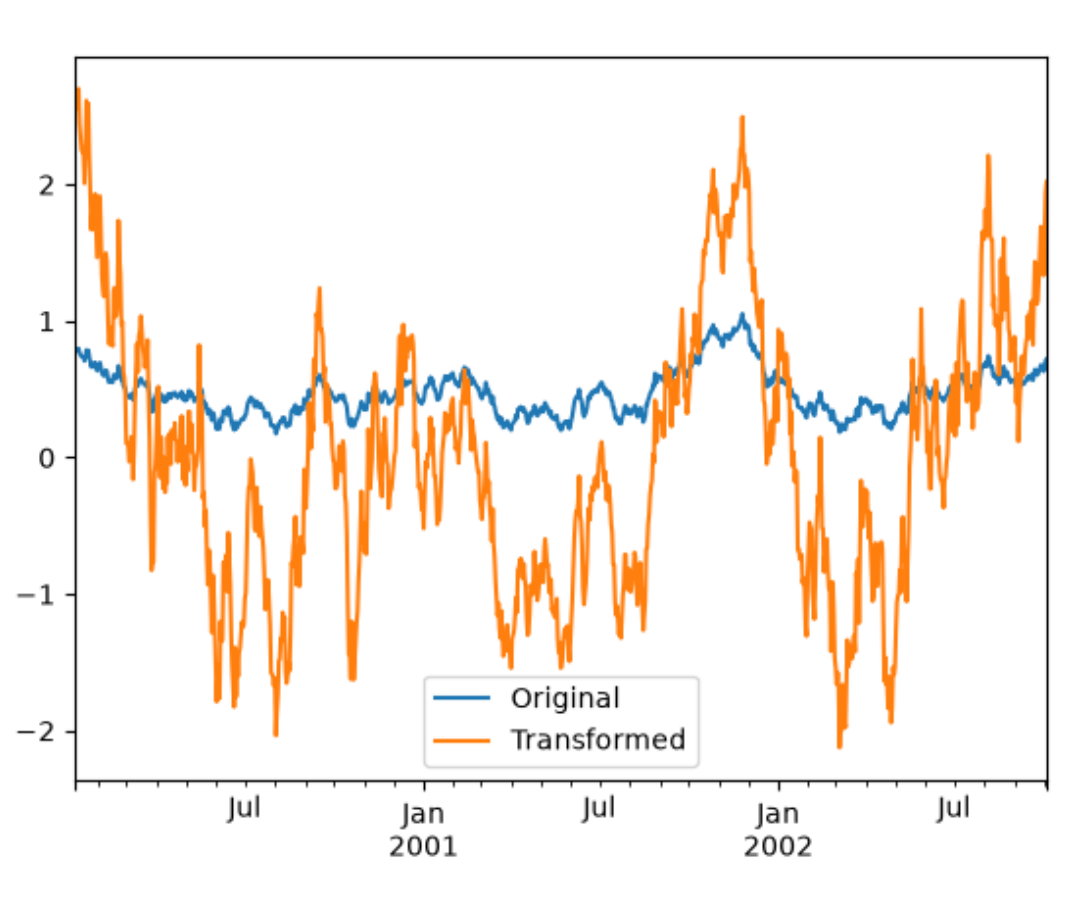
We would expect the result to now have mean 0 and standard deviation 1 within each group (up to floating-point error), which we can easily check:

```
# Original Data
In [134]: grouped = ts.groupby(lambda x: x.year)

In [135]: grouped.mean()
Out[135]: 
2000    0.442441
2001    0.526246
2002    0.459365
dtype: float64

In [136]: grouped.std()
Out[136]: 
2000    0.131752
2001    0.210945
```

```
2002    0.128753
dtype: float64

# Transformed Data
In [137]: grouped_trans = transformed.groupby(lambda x: x.year)

In [138]: grouped_trans.mean()
Out[138]:
2000   -4.870756e-16
2001   -1.545187e-16
2002    4.136282e-16
dtype: float64

In [139]: grouped_trans.std()
Out[139]:
2000    1.0
2001    1.0
2002    1.0
dtype: float64
```

We can also visually compare the original and transformed data sets.

```
In [140]: compare = pd.DataFrame({"Original": ts, "Transformed": transformed})

In [141]: compare.plot()
Out[141]: <Axes: >
```



Transformation functions that have lower dimension outputs are broadcast to match the shape of the input array.

```
In [142]: ts.groupby(lambda x: x.year).transform(lambda x: x.max() - x.min())
Out[142]:
2000-01-08    0.623893
2000-01-09    0.623893
2000-01-10    0.623893
2000-01-11    0.623893
2000-01-12    0.623893
                ...
2002-09-30    0.558275
2002-10-01    0.558275
2002-10-02    0.558275
2002-10-03    0.558275
2002-10-04    0.558275
Freq: D, Length: 1001, dtype: float64
```

Another common data transform is to replace missing data with the group mean.

```
In [143]: cols = ["A", "B", "C"]

In [144]: values = np.random.randn(1000, 3)

In [145]: values[np.random.randint(0, 1000, 100), 0] = np.nan

In [146]: values[np.random.randint(0, 1000, 50), 1] = np.nan

In [147]: values[np.random.randint(0, 1000, 200), 2] = np.nan

In [148]: data_df = pd.DataFrame(values, columns=cols)

In [149]: data_df
Out[149]:
            A         B         C
0    1.539708 -1.166480  0.533026
1    1.302092 -0.505754       NaN
2   -0.371983  1.104803 -0.651520
3   -1.309622  1.118697 -1.161657
4   -1.924296  0.396437  0.812436
..        ...       ...       ...
995 -0.093110  0.683847 -0.774753
996 -0.185043  1.438572       NaN
997 -0.394469 -0.642343  0.011374
998 -1.174126  1.857148       NaN
999  0.234564  0.517098  0.393534

[1000 rows x 3 columns]

In [150]: countries = np.array(["US", "UK", "GR", "JP"])

In [151]: key = countries[np.random.randint(0, 4, 1000)]

In [152]: grouped = data_df.groupby(key)

# Non-NA count in each group
In [153]: grouped.count()
Out[153]:
      A    B    C
GR  209  217  189
JP  240  255  217
UK  216  231  193
US  239  250  217

In [154]: transformed = grouped.transform(lambda x: x.fillna(x.mean()))
```

We can verify that the group means have not changed in the transformed data, and that the transformed data contains no NAs.

```
In [155]: grouped_trans = transformed.groupby(key)

In [156]: grouped.mean()  # original group means
Out[156]:
           A         B         C
GR -0.098371 -0.015420  0.068053
JP  0.069025  0.023100 -0.077324
UK  0.034069 -0.052580 -0.116525
US  0.058664 -0.020399  0.028603

In [157]: grouped_trans.mean()  # transformation did not change group means
Out[157]:
           A         B         C
GR -0.098371 -0.015420  0.068053
JP  0.069025  0.023100 -0.077324
UK  0.034069 -0.052580 -0.116525
US  0.058664 -0.020399  0.028603

In [158]: grouped.count()  # original has some missing data points
Out[158]:
      A    B    C
GR  209  217  189
JP  240  255  217
UK  216  231  193
US  239  250  217

In [159]: grouped_trans.count()  # counts after transformation
Out[159]:
      A    B    C
GR  228  228  228
JP  267  267  267
UK  247  247  247
US  258  258  258

In [160]: grouped_trans.size()  # Verify non-NA count equals group size
Out[160]:
GR    228
JP    267
UK    247
US    258
dtype: int64
```

As mentioned in the note above, each of the examples in this section can be computed more efficiently using built-in methods. In the code below, the inefficient way using a UDF is commented out and the faster alternative appears below.

```
# result = ts.groupby(lambda x: x.year).transform(
#     lambda x: (x - x.mean()) / x.std()
# )
In [161]: grouped = ts.groupby(lambda x: x.year)

In [162]: result = (ts - grouped.transform("mean")) / grouped.transform("std")

# result = ts.groupby(lambda x: x.year).transform(lambda x: x.max() - x.min())
In [163]: grouped = ts.groupby(lambda x: x.year)

In [164]: result = grouped.transform("max") - grouped.transform("min")

# grouped = data_df.groupby(key)
# result = grouped.transform(lambda x: x.fillna(x.mean()))
In [165]: grouped = data_df.groupby(key)

In [166]: result = data_df.fillna(grouped.transform("mean"))
```

Window and resample operations

It is possible to use `resample()`, `expanding()` and `rolling()` as methods on groupbys.

The example below will apply the `rolling()` method on the samples of the column B, based on the groups of column A.

```
In [167]: df_re = pd.DataFrame({"A": [1] * 10 + [5] * 10, "B": np.arange(20)})

In [168]: df_re
Out[168]:
    A   B
0   1   0
1   1   1
2   1   2
3   1   3
4   1   4
.. ..  ..
15  5  15
16  5  16
17  5  17
18  5  18
19  5  19

[20 rows x 2 columns]

In [169]: df_re.groupby("A").rolling(4).B.mean()
Out[169]:
A
1  0      NaN
   1      NaN
   2      NaN
   3      1.5
   4      2.5
         ...
5  15    13.5
   16    14.5
   17    15.5
   18    16.5
   19    17.5
Name: B, Length: 20, dtype: float64
```

The `expanding()` method will accumulate a given operation (`sum()` in the example) for all the members of each particular group.

```
In [170]: df_re.groupby("A").expanding().sum()
Out[170]:
          B
A
1 0     0.0
  1     1.0
  2     3.0
  3     6.0
  4    10.0
...     ...
5 15   75.0
  16   91.0
  17  108.0
  18  126.0
  19  145.0

[20 rows x 1 columns]
```

Suppose you want to use the `resample()` method to get a daily frequency in each group of your dataframe, and wish to complete the missing values with the `ffill()` method.

```
In [171]: df_re = pd.DataFrame(
   .....:     {
   .....:         "date": pd.date_range(start="2016-01-01", periods=4, freq="W"),
   .....:         "group": [1, 1, 2, 2],
   .....:         "val": [5, 6, 7, 8],
   .....:     }
   .....: ).set_index("date")
   .....:

In [172]: df_re
Out[172]:
            group  val
date
2016-01-03      1    5
2016-01-10      1    6
2016-01-17      2    7
2016-01-24      2    8

In [173]: df_re.groupby("group").resample("1D").ffill()
Out[173]:
                  val
group date
1     2016-01-03    5
      2016-01-04    5
      2016-01-05    5
      2016-01-06    5
      2016-01-07    5
...               ...
2     2016-01-20    7
      2016-01-21    7
      2016-01-22    7
      2016-01-23    7
      2016-01-24    8

[16 rows x 1 columns]
```

Filtration

A filtration is a GroupBy operation that subsets the original grouping object. It may either filter out entire groups, part of groups, or both. Filtrations return a filtered version of the calling object, including the grouping columns when provided. In the following example, `class` is included in the result.

```
In [174]: speeds
Out[174]:
          class           order  max_speed
falcon     bird   Falconiformes      389.0
parrot     bird  Psittaciformes       24.0
lion     mammal       Carnivora       80.2
monkey   mammal        Primates        NaN
leopard  mammal       Carnivora       58.0

In [175]: speeds.groupby("class").nth(1)
Out[175]:
         class           order  max_speed
parrot    bird  Psittaciformes       24.0
monkey  mammal        Primates        NaN
```

> **Note**
>
> Unlike aggregations, filtrations do not add the group keys to the index of the result. Because of this, passing `as_index=False` or `sort=True` will not affect these methods.

Filtrations will respect subsetting the columns of the GroupBy object.

```
In [176]: speeds.groupby("class")[["order", "max_speed"]].nth(1)
Out[176]:
                 order  max_speed
parrot  Psittaciformes       24.0
monkey        Primates        NaN
```

Built-in filtrations

The following methods on GroupBy act as filtrations. All these methods have an efficient, GroupBy-specific, implementation.

Method	Description
`head()`	Select the top row(s) of each group
`nth()`	Select the nth row(s) of each group
`tail()`	Select the bottom row(s) of each group

Users can also use transformations along with Boolean indexing to construct complex filtrations within groups. For example, suppose we are given groups of products and their volumes, and we wish to subset the data to only the largest products capturing no more than 90% of the total volume within each group.

```
In [177]: product_volumes = pd.DataFrame(
   .....:     {
   .....:         "group": list("xxxxyyy"),
   .....:         "product": list("abcdefg"),
   .....:         "volume": [10, 30, 20, 15, 40, 10, 20],
   .....:     }
   .....: )
   .....:

In [178]: product_volumes
Out[178]:
  group product  volume
0     x       a      10
1     x       b      30
2     x       c      20
3     x       d      15
4     y       e      40
5     y       f      10
6     y       g      20

# Sort by volume to select the largest products first
In [179]: product_volumes = product_volumes.sort_values("volume", ascending=False)

In [180]: grouped = product_volumes.groupby("group")["volume"]

In [181]: cumpct = grouped.cumsum() / grouped.transform("sum")

In [182]: cumpct
Out[182]:
4    0.571429
1    0.400000
2    0.666667
6    0.857143
3    0.866667
0    1.000000
5    1.000000
Name: volume, dtype: float64

In [183]: significant_products = product_volumes[cumpct <= 0.9]

In [184]: significant_products.sort_values(["group", "product"])
Out[184]:
  group product  volume
1     x       b      30
2     x       c      20
3     x       d      15
4     y       e      40
6     y       g      20
```

The `filter` method

> **Note**
>
> Filtering by supplying `filter` with a User-Defined Function (UDF) is often less performant than using the built-in methods on GroupBy. Consider breaking up a complex operation into a chain of operations that utilize the built-in methods.

The `filter` method takes a User-Defined Function (UDF) that, when applied to an entire group, returns either `True` or `False`. The result of the `filter` method is then the subset of groups for which the UDF returned `True`.

Suppose we want to take only elements that belong to groups with a group sum greater than 2.

```
In [185]: sf = pd.Series([1, 1, 2, 3, 3, 3])

In [186]: sf.groupby(sf).filter(lambda x: x.sum() > 2)
Out[186]:
3    3
4    3
5    3
dtype: int64
```

Another useful operation is filtering out elements that belong to groups with only a couple members.

```
In [187]: dff = pd.DataFrame({"A": np.arange(8), "B": list("aabbbbcc")})

In [188]: dff.groupby("B").filter(lambda x: len(x) > 2)
Out[188]:
   A  B
2  2  b
3  3  b
4  4  b
5  5  b
```

Alternatively, instead of dropping the offending groups, we can return a like-indexed objects where the groups that do not pass the filter are filled with NaNs.

```
In [189]: dff.groupby("B").filter(lambda x: len(x) > 2, dropna=False)
Out[189]:
     A    B
0  NaN  NaN
1  NaN  NaN
2  2.0    b
3  3.0    b
4  4.0    b
5  5.0    b
6  NaN  NaN
7  NaN  NaN
```

For DataFrames with multiple columns, filters should explicitly specify a column as the filter criterion.

```
In [190]: dff["C"] = np.arange(8)

In [191]: dff.groupby("B").filter(lambda x: len(x["C"]) > 2)
Out[191]:
   A  B  C
2  2  b  2
3  3  b  3
4  4  b  4
5  5  b  5
```

Flexible `apply`

Some operations on the grouped data might not fit into the aggregation, transformation, or filtration categories. For these, you can use the `apply` function.

> **Warning**
>
> `apply` has to try to infer from the result whether it should act as a reducer, transformer, *or* filter, depending on exactly what is passed to it. Thus the grouped column(s) may be included in the output or not. While it tries to intelligently guess how to behave, it can sometimes guess wrong.

> **Note**
>
> All of the examples in this section can be more reliably, and more efficiently, computed using other pandas functionality.

```
In [192]: df
Out[192]:
     A      B         C         D
0  foo    one -0.575247  1.346061
1  bar    one  0.254161  1.511763
2  foo    two -1.143704  1.627081
3  bar  three  0.215897 -0.990582
4  foo    two  1.193555 -0.441652
5  bar    two -0.077118  1.211526
6  foo    one -0.408530  0.268520
7  foo  three -0.862495  0.024580

In [193]: grouped = df.groupby("A")

# could also just call .describe()
In [194]: grouped["C"].apply(lambda x: x.describe())
Out[194]:
A
bar  count    3.000000
     mean     0.130980
     std      0.181231
     min     -0.077118
     25%      0.069390
                ...
foo  min     -1.143704
     25%     -0.862495
     50%     -0.575247
     75%     -0.408530
     max      1.193555
Name: C, Length: 16, dtype: float64
```

The dimension of the returned result can also change:

```
In [195]: grouped = df.groupby('A')['C']

In [196]: def f(group):
   .....:     return pd.DataFrame({'original': group,
   .....:                          'demeaned': group - group.mean()})
   .....:

In [197]: grouped.apply(f)
Out[197]:
       original  demeaned
A
bar 1  0.254161  0.123181
    3  0.215897  0.084917
    5 -0.077118 -0.208098
foo 0 -0.575247 -0.215962
    2 -1.143704 -0.784420
    4  1.193555  1.552839
    6 -0.408530 -0.049245
    7 -0.862495 -0.503211
```

`apply` on a Series can operate on a returned value from the applied function that is itself a series, and possibly upcast the result to a DataFrame:

```
In [198]: def f(x):
   .....:     return pd.Series([x, x ** 2], index=["x", "x^2"])
   .....:

In [199]: s = pd.Series(np.random.rand(5))

In [200]: s
Out[200]:
0    0.582898
1    0.098352
2    0.001438
3    0.009420
4    0.815826
dtype: float64

In [201]: s.apply(f)
Out[201]:
          x       x^2
0  0.582898  0.339770
1  0.098352  0.009673
2  0.001438  0.000002
3  0.009420  0.000089
4  0.815826  0.665572
```

Similar to The aggregate() method, the resulting dtype will reflect that of the apply function. If the results from different groups have different dtypes, then a common dtype will be determined in the same way as `DataFrame` construction.

Control grouped column(s) placement with `group_keys`

To control whether the grouped column(s) are included in the indices, you can use the argument `group_keys` which defaults to `True`. Compare

```
In [202]: df.groupby("A", group_keys=True).apply(lambda x: x)
Out[202]:
           B         C         D
A
bar 1    one  0.254161  1.511763
    3  three  0.215897 -0.990582
    5    two -0.077118  1.211526
foo 0    one -0.575247  1.346061
    2    two -1.143704  1.627081
    4    two  1.193555 -0.441652
    6    one -0.408530  0.268520
    7  three -0.862495  0.024580
```

with

```
In [203]: df.groupby("A", group_keys=False).apply(lambda x: x)
Out[203]:
       B         C         D
0    one -0.575247  1.346061
1    one  0.254161  1.511763
2    two -1.143704  1.627081
3  three  0.215897 -0.990582
4    two  1.193555 -0.441652
5    two -0.077118  1.211526
6    one -0.408530  0.268520
7  three -0.862495  0.024580
```

Numba accelerated routines

If Numba is installed as an optional dependency, the `transform` and `aggregate` methods support `engine='numba'` and `engine_kwargs` arguments. See enhancing performance with Numba for general usage of the arguments and performance considerations.

The function signature must start with `values, index` **exactly** as the data belonging to each group will be passed into `values`, and the group index will be passed into `index`.

> **Warning**
>
> When using `engine='numba'`, there will be no “fall back” behavior internally. The group data and group index will be passed as NumPy arrays to the JITed user defined function, and no alternative execution attempts will be tried.

Other useful features

Exclusion of non-numeric columns

Again consider the example DataFrame we’ve been looking at:

```
In [204]: df
Out[204]:
     A      B         C         D
0  foo    one -0.575247  1.346061
1  bar    one  0.254161  1.511763
2  foo    two -1.143704  1.627081
3  bar  three  0.215897 -0.990582
4  foo    two  1.193555 -0.441652
5  bar    two -0.077118  1.211526
6  foo    one -0.408530  0.268520
7  foo  three -0.862495  0.024580
```

Suppose we wish to compute the standard deviation grouped by the `A` column. There is a slight problem, namely that we don’t care about the data in column `B` because it is not numeric. You can avoid non-numeric columns by specifying `numeric_only=True`:

```
In [205]: df.groupby("A").std(numeric_only=True)
Out[205]:
            C         D
A
bar  0.181231  1.366330
foo  0.912265  0.884785
```

Note that `df.groupby('A').colname.std().` is more efficient than `df.groupby('A').std().colname`. So if the result of an aggregation function is only needed over one column (here `colname`), it may be filtered *before* applying the aggregation function.

```
In [206]: from decimal import Decimal

In [207]: df_dec = pd.DataFrame(
   .....:     {
   .....:         "id": [1, 2, 1, 2],
   .....:         "int_column": [1, 2, 3, 4],
   .....:         "dec_column": [
   .....:             Decimal("0.50"),
   .....:             Decimal("0.15"),
   .....:             Decimal("0.25"),
   .....:             Decimal("0.40"),
   .....:         ],
   .....:     }
   .....: )
   .....:

In [208]: df_dec.groupby(["id"])[["dec_column"]].sum()
Out[208]:
   dec_column
id
1        0.75
2        0.55
```

Handling of (un)observed Categorical values

When using a `Categorical` grouper (as a single grouper, or as part of multiple groupers), the `observed` keyword controls whether to return a cartesian product of all possible groupers values (`observed=False`) or only those that are observed groupers (`observed=True`).

Show all values:

```
In [209]: pd.Series([1, 1, 1]).groupby(
   .....:     pd.Categorical(["a", "a", "a"], categories=["a", "b"]), observed=Fals
   .....: ).count()
   .....:
Out[209]:
a    3
b    0
dtype: int64
```

Show only the observed values:

```
In [210]: pd.Series([1, 1, 1]).groupby(
   .....:     pd.Categorical(["a", "a", "a"], categories=["a", "b"]), observed=True
   .....: ).count()
   .....:
Out[210]:
a    3
dtype: int64
```

The returned dtype of the grouped will *always* include *all* of the categories that were grouped.

```
In [211]: s = (
   .....:     pd.Series([1, 1, 1])
   .....:     .groupby(pd.Categorical(["a", "a", "a"], categories=["a", "b"]), obse
   .....:     .count()
   .....: )
   .....:

In [212]: s.index.dtype
Out[212]: CategoricalDtype(categories=['a', 'b'], ordered=False, categories_dtype=s
```

NA group handling

By `NA`, we are referring to any `NA` values, including `NA`, `NaN`, `NaT`, and `None`. If there are any `NA` values in the grouping key, by default these will be excluded. In other words, any “`NA` group” will be dropped. You can include NA groups by specifying `dropna=False`.

```
In [213]: df = pd.DataFrame({"key": [1.0, 1.0, np.nan, 2.0, np.nan], "A": [1, 2, 3,

In [214]: df
Out[214]:
   key  A
0  1.0  1
1  1.0  2
2  NaN  3
3  2.0  4
4  NaN  5

In [215]: df.groupby("key", dropna=True).sum()
Out[215]:
     A
key
1.0  3
2.0  4

In [216]: df.groupby("key", dropna=False).sum()
Out[216]:
     A
key
1.0  3
2.0  4
NaN  8
```

Grouping with ordered factors

Categorical variables represented as instances of pandas's `Categorical` class can be used as group keys. If so, the order of the levels will be preserved. When `observed=False` and `sort=False`, any unobserved categories will be at the end of the result in order.

```
In [217]: days = pd.Categorical(
   .....:     values=["Wed", "Mon", "Thu", "Mon", "Wed", "Sat"],
   .....:     categories=["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"],
   .....: )
   .....:

In [218]: data = pd.DataFrame(
   .....:    {
   .....:        "day": days,
   .....:        "workers": [3, 4, 1, 4, 2, 2],
   .....:    }
   .....: )
   .....:

In [219]: data
Out[219]:
   day  workers
0  Wed        3
1  Mon        4
2  Thu        1
3  Mon        4
4  Wed        2
5  Sat        2

In [220]: data.groupby("day", observed=False, sort=True).sum()
Out[220]:
     workers
day
Mon        8
Tue        0
Wed        5
Thu        1
Fri        0
Sat        2
Sun        0

In [221]: data.groupby("day", observed=False, sort=False).sum()
Out[221]:
     workers
day
Wed        5
Mon        8
Thu        1
Sat        2
Tue        0
Fri        0
Sun        0
```

Grouping with a grouper specification

You may need to specify a bit more data to properly group. You can use the `pd.Grouper` to provide this local control.

```
In [222]: import datetime

In [223]: df = pd.DataFrame(
   .....:     {
   .....:         "Branch": "A A A A A A A B".split(),
   .....:         "Buyer": "Carl Mark Carl Carl Joe Joe Joe Carl".split(),
   .....:         "Quantity": [1, 3, 5, 1, 8, 1, 9, 3],
   .....:         "Date": [
   .....:             datetime.datetime(2013, 1, 1, 13, 0),
   .....:             datetime.datetime(2013, 1, 1, 13, 5),
   .....:             datetime.datetime(2013, 10, 1, 20, 0),
   .....:             datetime.datetime(2013, 10, 2, 10, 0),
   .....:             datetime.datetime(2013, 10, 1, 20, 0),
   .....:             datetime.datetime(2013, 10, 2, 10, 0),
   .....:             datetime.datetime(2013, 12, 2, 12, 0),
   .....:             datetime.datetime(2013, 12, 2, 14, 0),
   .....:         ],
   .....:     }
   .....: )
   .....:

In [224]: df
Out[224]:
  Branch Buyer  Quantity                Date
0      A  Carl         1 2013-01-01 13:00:00
1      A  Mark         3 2013-01-01 13:05:00
2      A  Carl         5 2013-10-01 20:00:00
3      A  Carl         1 2013-10-02 10:00:00
4      A   Joe         8 2013-10-01 20:00:00
5      A   Joe         1 2013-10-02 10:00:00
6      A   Joe         9 2013-12-02 12:00:00
7      B  Carl         3 2013-12-02 14:00:00
```

Groupby a specific column with the desired frequency. This is like resampling.

```
In [225]: df.groupby([pd.Grouper(freq="1ME", key="Date"), "Buyer"])[["Quantity"]].s
Out[225]:
                  Quantity
Date       Buyer
2013-01-31 Carl          1
           Mark          3
2013-10-31 Carl          6
           Joe           9
2013-12-31 Carl          3
           Joe           9
```

When `freq` is specified, the object returned by `pd.Grouper` will be an instance of `pandas.api.typing.TimeGrouper`. When there is a column and index with the same name, you can use `key` to group by the column and `level` to group by the index.

```
In [226]: df = df.set_index("Date")

In [227]: df["Date"] = df.index + pd.offsets.MonthEnd(2)

In [228]: df.groupby([pd.Grouper(freq="6ME", key="Date"), "Buyer"])[["Quantity"]].s
Out[228]:
                  Quantity
Date       Buyer
2013-02-28 Carl          1
           Mark          3
2014-02-28 Carl          6
           Joe          18

In [229]: df.groupby([pd.Grouper(freq="6ME", level="Date"), "Buyer"])[["Quantity"]]
Out[229]:
                  Quantity
Date       Buyer
2013-01-31 Carl          1
           Mark          3
2014-01-31 Carl          9
           Joe          18
```

Taking the first rows of each group

Just like for a DataFrame or Series you can call head and tail on a groupby:

```
In [230]: df = pd.DataFrame([[1, 2], [1, 4], [5, 6]], columns=["A", "B"])

In [231]: df
Out[231]:
   A  B
0  1  2
1  1  4
2  5  6

In [232]: g = df.groupby("A")

In [233]: g.head(1)
Out[233]:
   A  B
0  1  2
2  5  6

In [234]: g.tail(1)
Out[234]:
   A  B
1  1  4
2  5  6
```

This shows the first or last n rows from each group.

Taking the nth row of each group

To select the nth item from each group, use **`DataFrameGroupBy.nth()`** or **`SeriesGroupBy.nth()`**. Arguments supplied can be any integer, lists of integers, slices, or lists of slices; see below for examples. When the nth element of a group does not exist an error is *not* raised; instead no corresponding rows are returned.

In general this operation acts as a filtration. In certain cases it will also return one row per group, making it also a reduction. However because in general it can return zero or multiple rows per group, pandas treats it as a filtration in all cases.

```
In [235]: df = pd.DataFrame([[1, np.nan], [1, 4], [5, 6]], columns=["A", "B"])

In [236]: g = df.groupby("A")

In [237]: g.nth(0)
Out[237]:
   A    B
0  1  NaN
2  5  6.0

In [238]: g.nth(-1)
Out[238]:
   A    B
1  1  4.0
2  5  6.0

In [239]: g.nth(1)
Out[239]:
   A    B
1  1  4.0
```

If the nth element of a group does not exist, then no corresponding row is included in the result. In particular, if the specified `n` is larger than any group, the result will be an empty DataFrame.

```
In [240]: g.nth(5)
Out[240]:
Empty DataFrame
Columns: [A, B]
Index: []
```

If you want to select the nth not-null item, use the `dropna` kwarg. For a DataFrame this should be either `'any'` or `'all'` just like you would pass to dropna:

```
# nth(0) is the same as g.first()
In [241]: g.nth(0, dropna="any")
Out[241]:
   A    B
1  1  4.0
2  5  6.0

In [242]: g.first()
Out[242]:
     B
A
1  4.0
5  6.0

# nth(-1) is the same as g.last()
In [243]: g.nth(-1, dropna="any")
Out[243]:
   A    B
1  1  4.0
2  5  6.0

In [244]: g.last()
Out[244]:
     B
A
1  4.0
5  6.0

In [245]: g.B.nth(0, dropna="all")
Out[245]:
1    4.0
2    6.0
Name: B, dtype: float64
```

You can also select multiple rows from each group by specifying multiple nth values as a list of ints.

```
In [246]: business_dates = pd.date_range(start="4/1/2014", end="6/30/2014", freq="B

In [247]: df = pd.DataFrame(1, index=business_dates, columns=["a", "b"])

# get the first, 4th, and last date index for each month
In [248]: df.groupby([df.index.year, df.index.month]).nth([0, 3, -1])
Out[248]:
            a  b
2014-04-01  1  1
2014-04-04  1  1
2014-04-30  1  1
2014-05-01  1  1
2014-05-06  1  1
2014-05-30  1  1
2014-06-02  1  1
2014-06-05  1  1
2014-06-30  1  1
```

You may also use slices or lists of slices.

```
In [249]: df.groupby([df.index.year, df.index.month]).nth[1:]
Out[249]:
            a  b
2014-04-02  1  1
2014-04-03  1  1
2014-04-04  1  1
2014-04-07  1  1
2014-04-08  1  1
...        .. ..
2014-06-24  1  1
2014-06-25  1  1
2014-06-26  1  1
2014-06-27  1  1
2014-06-30  1  1

[62 rows x 2 columns]

In [250]: df.groupby([df.index.year, df.index.month]).nth[1:, :-1]
Out[250]:
            a  b
2014-04-01  1  1
2014-04-02  1  1
2014-04-03  1  1
2014-04-04  1  1
2014-04-07  1  1
...        .. ..
2014-06-24  1  1
2014-06-25  1  1
2014-06-26  1  1
2014-06-27  1  1
2014-06-30  1  1

[65 rows x 2 columns]
```

Enumerate group items

To see the order in which each row appears within its group, use the `cumcount` method:

```
In [251]: dfg = pd.DataFrame(list("aaabba"), columns=["A"])

In [252]: dfg
Out[252]:
   A
0  a
1  a
2  a
3  b
4  b
5  a

In [253]: dfg.groupby("A").cumcount()
Out[253]:
0    0
1    1
2    2
3    0
4    1
5    3
dtype: int64

In [254]: dfg.groupby("A").cumcount(ascending=False)
Out[254]:
0    3
1    2
2    1
3    1
4    0
5    0
dtype: int64
```

Enumerate groups

To see the ordering of the groups (as opposed to the order of rows within a group given by `cumcount`) you can use **`DataFrameGroupBy.ngroup()`**.

Note that the numbers given to the groups match the order in which the groups would be seen when iterating over the groupby object, not the order they are first observed.

```
In [255]: dfg = pd.DataFrame(list("aaabba"), columns=["A"])

In [256]: dfg
Out[256]:
   A
0  a
1  a
2  a
3  b
4  b
5  a

In [257]: dfg.groupby("A").ngroup()
Out[257]:
0    0
1    0
2    0
3    1
4    1
5    0
dtype: int64

In [258]: dfg.groupby("A").ngroup(ascending=False)
Out[258]:
0    1
1    1
2    1
3    0
4    0
5    1
dtype: int64
```

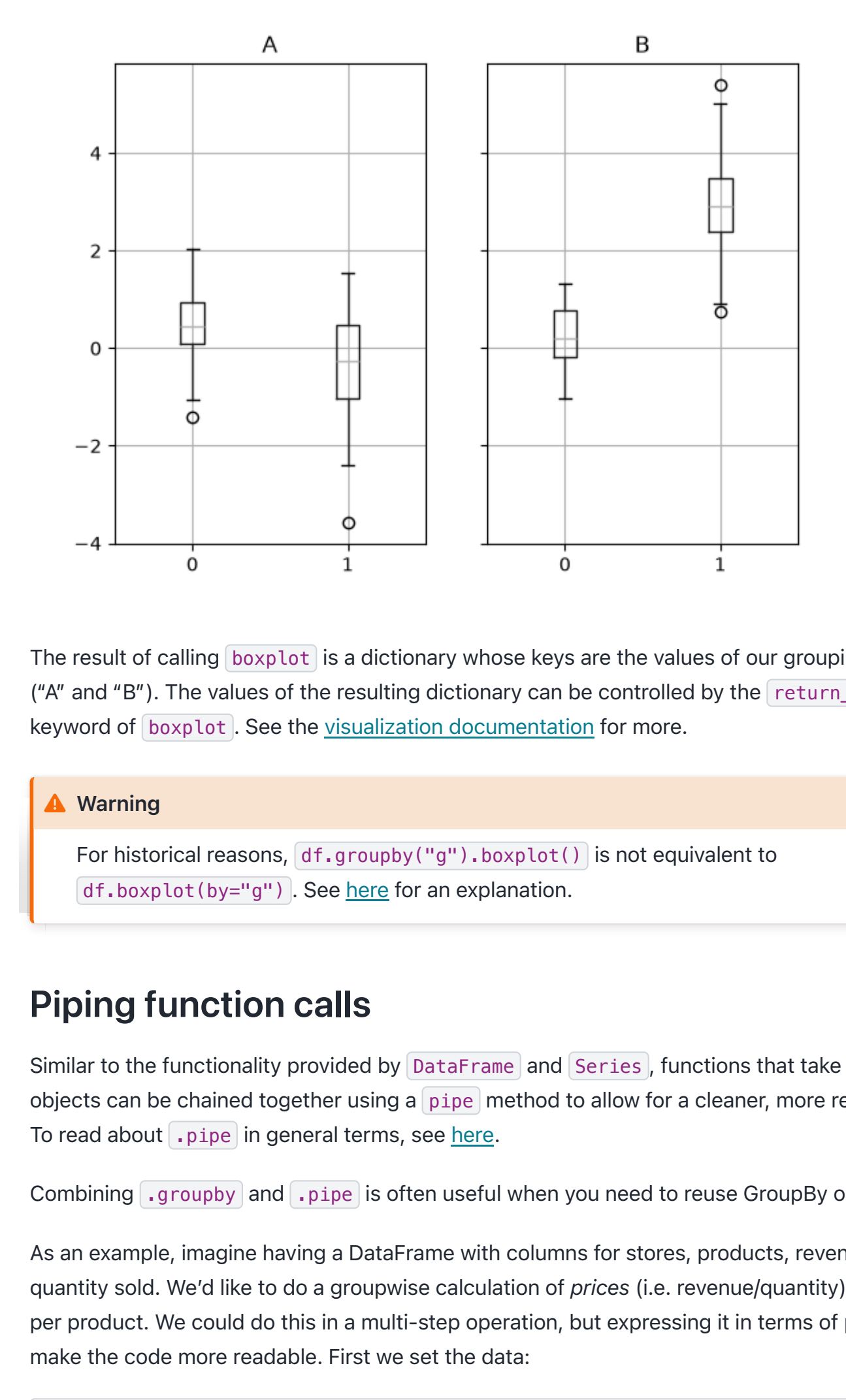
Plotting

Groupby also works with some plotting methods. In this case, suppose we suspect that the values in column 1 are 3 times higher on average in group "B".

```
In [259]: np.random.seed(1234)

In [260]: df = pd.DataFrame(np.random.randn(50, 2))

In [261]: df["g"] = np.random.choice(["A", "B"], size=50)

In [262]: df.loc[df["g"] == "B", 1] += 3
```

We can easily visualize this with a boxplot:

```
In [263]: df.groupby("g").boxplot()
Out[263]:
A         Axes(0.1,0.15;0.363636x0.75)
B    Axes(0.536364,0.15;0.363636x0.75)
dtype: object
```



The result of calling `boxplot` is a dictionary whose keys are the values of our grouping column `g` ("A" and "B"). The values of the resulting dictionary can be controlled by the `return_type` keyword of `boxplot`. See the visualization documentation for more.

Warning

For historical reasons, `df.groupby("g").boxplot()` is not equivalent to `df.boxplot(by="g")`. See here for an explanation.

Piping function calls

Similar to the functionality provided by `DataFrame` and `Series`, functions that take `GroupBy` objects can be chained together using a `pipe` method to allow for a cleaner, more readable syntax. To read about `.pipe` in general terms, see here.

Combining `.groupby` and `.pipe` is often useful when you need to reuse GroupBy objects.

As an example, imagine having a DataFrame with columns for stores, products, revenue and quantity sold. We'd like to do a groupwise calculation of *prices* (i.e. revenue/quantity) per store and per product. We could do this in a multi-step operation, but expressing it in terms of piping can make the code more readable. First we set the data:

```
In [264]: n = 1000

In [265]: df = pd.DataFrame(
   .....:     {
   .....:         "Store": np.random.choice(["Store_1", "Store_2"], n),
   .....:         "Product": np.random.choice(["Product_1", "Product_2"], n),
   .....:         "Revenue": (np.random.random(n) * 50 + 10).round(2),
   .....:         "Quantity": np.random.randint(1, 10, size=n),
   .....:     }
   .....: )
   .....:

In [266]: df.head(2)
Out[266]:
     Store    Product  Revenue  Quantity
0  Store_2  Product_1    26.12         1
1  Store_2  Product_1    28.86         1
```

We now find the prices per store/product.

```
In [267]: (
   .....:     df.groupby(["Store", "Product"])
   .....:     .pipe(lambda grp: grp.Revenue.sum() / grp.Quantity.sum())
   .....:     .unstack()
   .....:     .round(2)
   .....: )
   .....:
Out[267]:
Product  Product_1  Product_2
Store
Store_1       6.82       7.05
Store_2       6.30       6.64
```

Piping can also be expressive when you want to deliver a grouped object to some arbitrary function, for example:

```
In [268]: def mean(groupby):
   .....:     return groupby.mean()
   .....:

In [269]: df.groupby(["Store", "Product"]).pipe(mean)
Out[269]:
                     Revenue  Quantity
Store   Product
Store_1 Product_1  34.622727  5.075758
        Product_2  35.482815  5.029630
Store_2 Product_1  32.972837  5.237589
        Product_2  34.684360  5.224000
```

Here `mean` takes a GroupBy object and finds the mean of the Revenue and Quantity columns respectively for each Store-Product combination. The `mean` function can be any function that takes in a GroupBy object; the `.pipe` will pass the GroupBy object as a parameter into the function you specify.

Examples

Multi-column factorization

By using **`DataFrameGroupBy.ngroup()`**, we can extract information about the groups in a way similar to **`factorize()`** (as described further in the reshaping API) but which applies naturally to multiple columns of mixed type and different sources. This can be useful as an intermediate categorical-like step in processing, when the relationships between the group rows are more important than their content, or as input to an algorithm which only accepts the integer encoding. (For more information about support in pandas for full categorical data, see the Categorical introduction and the API documentation.)

```
In [270]: dfg = pd.DataFrame({"A": [1, 1, 2, 3, 2], "B": list("aaaba")})

In [271]: dfg
Out[271]:
   A  B
0  1  a
1  1  a
2  2  a
3  3  b
4  2  a

In [272]: dfg.groupby(["A", "B"]).ngroup()
Out[272]:
0    0
1    0
2    1
3    2
4    1
dtype: int64

In [273]: dfg.groupby(["A", [0, 0, 0, 1, 1]]).ngroup()
Out[273]:
0    0
1    0
2    1
3    3
4    2
dtype: int64
```

GroupBy by indexer to 'resample' data

Resampling produces new hypothetical samples (resamples) from already existing observed data or from a model that generates data. These new samples are similar to the pre-existing samples.

In order for resample to work on indices that are non-datetimelike, the following procedure can be utilized.

In the following examples, **df.index // 5** returns an integer array which is used to determine what gets selected for the groupby operation.

Note

The example below shows how we can downsample by consolidation of samples into fewer ones. Here by using **df.index // 5**, we are aggregating the samples in bins. By applying **std()** function, we aggregate the information contained in many samples into a small subset of values which is their standard deviation thereby reducing the number of samples.

```
In [274]: df = pd.DataFrame(np.random.randn(10, 2))

In [275]: df
Out[275]:
          0         1
0 -0.793893  0.321153
1  0.342250  1.618906
2 -0.975807  1.918201
3 -0.810847 -1.405919
4 -1.977759  0.461659
5  0.730057 -1.316938
6 -0.751328  0.528290
7 -0.257759 -1.081009
8  0.505895 -1.701948
9 -1.006349  0.020208

In [276]: df.index // 5
Out[276]: Index([0, 0, 0, 0, 0, 1, 1, 1, 1, 1], dtype='int64')

In [277]: df.groupby(df.index // 5).std()
Out[277]:
          0         1
0  0.823647  1.312912
1  0.760109  0.942941
```

Returning a Series to propagate names

Group DataFrame columns, compute a set of metrics and return a named Series. The Series name is used as the name for the column index. This is especially useful in conjunction with reshaping operations such as stacking, in which the column index name will be used as the name of the inserted column:

```
In [278]: df = pd.DataFrame(
   .....:     {
   .....:         "a": [0, 0, 0, 0, 1, 1, 1, 1, 2, 2, 2, 2],
   .....:         "b": [0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1],
   .....:         "c": [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0],
   .....:         "d": [0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1],
   .....:     }
   .....: )
   .....:

In [279]: def compute_metrics(x):
   .....:     result = {"b_sum": x["b"].sum(), "c_mean": x["c"].mean()}
   .....:     return pd.Series(result, name="metrics")
   .....:

In [280]: result = df.groupby("a").apply(compute_metrics)

In [281]: result
Out[281]:
metrics  b_sum  c_mean
a
0          2.0     0.5
1          2.0     0.5
2          2.0     0.5

In [282]: result.stack()
Out[282]:
a  metrics
0  b_sum      2.0
   c_mean     0.5
1  b_sum      2.0
   c_mean     0.5
2  b_sum      2.0
   c_mean     0.5
dtype: float64
```

Previous
User-Defined Functions (UDFs)

Next
Windowing operations

© 2026, pandas via NumFOCUS, Inc. Hosted by OVHcloud.

Created using Sphinx 9.0.4.

Built with the PyData Sphinx Theme 0.16.1.